Project Proposal: Detecting Fraud in Financial Transactions

Krishna Damarla

College of Graduate and Professional Studies

Trine University

IS 5113: Data Mining and Data Visualization

Dr. Anas Al-Tirawi

September 14, 2025

Project Proposal: Detecting Fraud in Financial Transactions

Bank fraud is a pressing problem for both financial institutions and customers. Traditional banking systems are often insufficient to combat evolving fraud tactics. This project proposal outlines steps to develop a real-time fraud detection system using the Cross-Industry Standard Process for Data Mining (CRISP-DM) framework, from business understanding to deployment (Delen, 2020). With advanced machine learning, the system aims to improve fraud detection rates while minimizing false positives to protect assets and enhance customer trust.

Problem Definition

Bank fraud is a big problem. It drains money, erodes trust, and can damage reputations. We need to spot fake transactions in real-time, before money leaves an account. If we block too many good purchases, customers get angry. If we miss the bad ones, both the bank and its customers lose money. We want to spot suspicious transactions fast, so we can protect customers and keep the bank's name clean (McCormick, 2024).

Objective

The goal is to build a model that flags likely fraud with high accuracy and can spot 90% of fraud in real-time while keeping false alarms under 1%. Such a model can help us to act quickly, reduce losses, and reassure customers that their money is safe.

CRISP-DM Framework Application

The six phases of CRISP-DM keep the project on track. Let's dive into each one of them.

Business Understanding

Banks needs to catch fraud quick and smart. The current system follows old rules and misses a lot of fraud. Hence, we aim to build a system that can catch 90% of fraud before it happens and make decisions in under 150 milliseconds. The system also needs to give clear

reasons for why a transaction is flagged and keep false alarms under 1%. This keeps regulators and customers happy.

Data Understanding

We collect data from different sources, such as transaction logs, customer profiles, device information, and location (e.g., IP address). The data includes numbers (e.g., transaction amounts), categories (e.g., account type), and time stamps.

Later, we explore the data, look for missing values, odd patterns, and anything that stands out from the transaction amount, time, place, customer spending patterns and merchant profiles. A quick peek into such sample data shows that fraud often happens at night for higher amounts, and only about 1 in 1000 transactions is fraud (McCormick, 2024).

Data Preparation

This is a key part as the model performance depends on data we supply to it. Four data preprocessing methods are applied to the sample dataset of bank transactions as detailed below.

Data Consolidation

Start by bringing all data from different sources, such as transaction databases, customer records, and device logs. For example, write SQL queries to join the transaction history table with the customer profiles table. The goal is one clean, unified dataset (Delen, 2020).

Data Cleaning

Here, you remove or smooth out all errors to create a cleaned dataset for further steps. For example, fill in missing values (purchase location with customer's home city), spot outliers ($1 million charge for a coffee), and fix errors (transaction amount way higher than normal).

Data Transformation

This step makes the data easier for the model to understand. For example, it converts all purchase amounts into US dollars, scale or normalize numbers to a common range (like 0 to 1), encode categorical variables or group transaction times into buckets (morning, afternoon, night). Also, create new features, such as "average spend per day" or "transaction frequency", as part of feature engineering to improve model performance (Murel, n.d.).

Data Reduction

You might have hundreds of data points for each transaction, but not all of them are useful. It's best to keep only the most useful features using methods like Principal Component Analysis (PCA) to focus on the top factors linked to fraud (Murel, n.d.).

Modeling

This is where we build the fraud-detecting system. We select a classification model such as random forest or gradient boost tree. These algorithms are well-suited for fraud-detection due to their high predictive accuracy in answering binary classification tasks like "Is this transaction fraudulent or legitimate?". The model choice is also to be based on interpretability and computational efficiency. After selecting the model algorithm(s), we train it on the prepared data.

Evaluation

How do we know if our model is any good? We test it by splitting the data into three sets. The training dataset (e.g., 70%) is where the model learns from. Validation dataset (e.g., 15%) fine tunes the model at the end of each training cycle or epoch or, at the end of the training to prevent overfitting. The test dataset (e.g., 15%) is to test the model on data it has never seen.

The model's decision is evaluated through metrics like precision, recall, F1-score, ROC-AUC and cross-validation. These metrics helps us balance false positives and false negatives.

They tell us how many of the flagged transactions were actually fraud, and how many real fraud cases were successfully caught by the model (Delen, 2020).

Deployment

Once we have a trusted model, we take it live by deploying it on docker. We roll it out carefully by running it in "shadow mode," where it makes predictions but doesn't act on them. This lets you watch its performance without any real-world risk. Then, you can slowly start letting it make real decisions for a small part of your transactions. Once everything is working as expected, you also need to keep monitoring and updating the model, as fraudsters are always changing their tactics.

Monitoring will include periodic retraining, incremental training on new data, performance tracking, and alerting for model drift or degradation. The final model will be integrated into the bank's transaction processing system, with real-time scoring of incoming transactions (McCormick, 2024).

References

Delen, D. (2020). Predictive analytics: Data mining, machine learning, and data science for practitioners (2nd ed.). *Pearson*.

McCormick, K. (2024). Predictive analytics essential training: Data mining. *LinkedIn Learning*. https://www.linkedin.com/learning/predictive-analytics-essential-training-data-mining/data-mining-and-predictive-analytics-24925114

Murel, J. (n.d.). Feature engineering. *IBM*. https://www.ibm.com/think/topics/feature-engineering